

Rapport Diagnostic C2A-Vente

Analyse statique du projet realisee le 06/05/2026. Aucun fichier source n'a ete modifie, aucune correction automatique n'a ete lancee, et ce PDF est le seul artefact ajoute.

1. Synthese

Le projet C2A-Vente est une application web full-stack de gestion commerciale. Il est organise en deux sous-projets distincts : un frontend React/Vite et un backend Spring Boot expose sous forme d API REST securisee par JWT. La base de donnees cible est MySQL.

- Architecture globale : three-tier, avec frontend React, backend Spring Boot et base MySQL.
- Frontend : React 18, Vite, Tailwind CSS, Axios, Recharts et react-hot-toast.
- Backend : Java 17, Spring Boot 3.2, Spring Security, JWT, Spring Data JPA, Lombok, Springdoc OpenAPI.
- API backend : contexte /api, port 8081.
- Frontend dev : port 3000, proxy /api vers http://localhost:8081.
- Etat general : structure claire, mais plusieurs risques de coherence, securite, numerotation et donnees dashboard.

2. Architecture Reelle

Frontend :	c2a-frontend - application React/Vite
Backend :	c2a-backend - application Spring Boot
Base de donnees :	MySQL, schema configure : c2a_vente
Authentification :	JWT Bearer, token stocke dans localStorage cote frontend
Routage frontend :	react-router-dom avec routes privees
Communication :	Axios baseUrl /api, proxy Vite vers backend

3. Role De Chaque Dossier

- c2a-frontend/src/api : configuration Axios et services API centralises.
- c2a-frontend/src/context : contexte AuthContext pour login, logout, stockage du user et token.
- c2a-frontend/src/components/layout : structure generale de l application, sidebar, topbar et outlet.
- c2a-frontend/src/components/ui : composants UI reutilisables : Modal, Badge, EmptyState, SearchInput, PageHeader, FormField.
- c2a-frontend/src/pages : pages metier React : auth, dashboard, clients, produits, devis, commandes, factures, stock.
- c2a-backend/src/main/java/.../controller : controleurs REST et mapping des endpoints.
- c2a-backend/src/main/java/.../service/impl : logique metier des modules clients, produits, devis, commandes, factures, stock, auth.
- c2a-backend/src/main/java/.../repository : interfaces Spring Data JPA.
- c2a-backend/src/main/java/.../entity : entites JPA et relations base de donnees.
- c2a-backend/src/main/java/.../dto : objets de requete/reponse exposes par API.
- c2a-backend/src/main/java/.../security : generation/validation JWT et filtre de securite.

- c2a-backend/src/main/java/.../config : configuration securite et initialisation des utilisateurs demo.
- c2a-backend/src/main/resources : application.properties.
- dist, target, node_modules : artefacts generes ou dependances locales, pas du code source applicatif.

4. Frontend React

Le frontend est concentre et lisible. Les appels HTTP sont regroupes dans services.js, les erreurs et le token sont geres par axios.js, et les routes privees sont definies dans App.jsx.

- /login : page de connexion.
- / : dashboard.
- /clients : gestion clients.
- /produits : catalogue produits.
- /devis : creation et suivi des devis, conversion en commande.
- /commandes : suivi et changement de statut des commandes.
- /factures : consultation factures et enregistrement des paiements.
- /stock : consultation stock, alertes rupture, mouvements et historique.

Fichiers frontend importants :

c2a-frontend/src/App.jsx : routes privees et mapping des pages.

c2a-frontend/src/main.jsx : BrowserRouter, Toaster et rendu React.

c2a-frontend/src/api/axios.js : baseURL /api, token Bearer, gestion 401/403.

c2a-frontend/src/api/services.js : clientsAPI, produitsAPI, devisAPI, commandesAPI, facturesAPI, stockAPI, dashboardAPI.

c2a-frontend/src/context/AuthContext.jsx : login, logout, user, loading, hasRole.

c2a-frontend/src/components/layout/Layout.jsx : sidebar et navigation globale.

c2a-frontend/src/components/ui/index.jsx : composants UI partages.

5. Backend Spring Boot

Le backend expose une API REST sous /api. La securite est stateless avec JWT. Les permissions sont definies par annotations @PreAuthorize sur plusieurs endpoints sensibles.

- AuthController : login et register.
- ClientController : lecture, creation, modification, desactivation et creances.
- ProduitController : lecture, creation, modification, produits en rupture.
- DevisController : creation, changement de statut, conversion en commande.
- CommandeController : lecture, filtrage site/statut, creation, changement de statut.
- FactureController : lecture, creation facture, factures en retard, paiements.
- StockController : mouvements stock, ruptures, enregistrement mouvement.
- DashboardController : indicateurs financiers et ventes par mois.

Fichiers backend importants :

c2a-backend/pom.xml : dependances Java/Spring.

c2a-backend/src/main/resources/application.properties : port, DB, JWT, Swagger, mail.

c2a-backend/src/main/java/com/c2a/gestionventes/config/SecurityConfig.java : Spring Security.

c2a-backend/src/main/java/com/c2a/gestionventes/security/JwtUtil.java : generation/validation JWT.

c2a-backend/src/main/java/com/c2a/gestionventes/security/JwtAuthFilter.java : extraction Bearer token.

c2a-backend/src/main/java/com/c2a/gestionventes/dto/BusinessDTOs.java : contrats metier.

c2a-backend/src/main/java/com/c2a/gestionventes/dto/AuthDTOs.java : contrats auth.

c2a-backend/src/main/java/com/c2a/gestionventes/entity/*.java : modele de donnees.

c2a-backend/src/main/java/com/c2a/gestionventes/repository/*.java : acces JPA.

c2a-backend/src/main/java/com/c2a/gestionventes/service/impl/*.java : logique metier.

6. Base De Donnees

La base est relationnelle et pilotée par JPA/Hibernate avec ddl-auto=update. Les principales tables proviennent des entités JPA.

- users : utilisateurs, roles, site, actif, credentials BCrypt.
- clients : identite client, contact, site, creditMax, soldeCreance, actif.
- produits : reference, designation, categorie, prix, unite, stockActuel, stockMin.
- devis et lignes_devis : documents commerciaux avant commande.
- commandes et lignes_commande : commandes client, montants, statut, livraison.
- factures : facture liee a commande, montant paye/restant, echeance, statut.
- paiements : paiements lies aux factures.
- mouvements_stock : historique des entrees, sorties, retours et ajustements.
- bons_livraison : modele present mais aucun controleur dedie visible dans le frontend actuel.

7. Securite Et Roles

Le backend definit six roles et applique les permissions cote serveur. Le frontend possede une fonction hasRole mais ne semble pas masquer les menus ni bloquer les pages par role.

- Roles : GERANT, ADMIN, COMMERCIAL, MAGASINIER, COMPTABLE, AGENT_RECOUVREMENT.
- Endpoints publics : /auth/**, /swagger-ui/**, /v3/api-docs/**.
- Toutes les autres routes backend demandent une authentication.
- JWT : role inclus dans les claims, expiration configuree a 86400000 ms.
- Frontend : token et user sauvegardes dans localStorage sous c2a_token et c2a_user.
- Risque : localStorage expose le token en cas de XSS.
- Risque : proprietes sensibles en clair dans application.properties.

8. Endpoints Principaux

AUTH

POST /api/auth/login

POST /api/auth/register

CLIENTS

GET /api/clients

GET /api/clients/{id}

GET /api/clients/site/{site}

GET /api/clients/creances

POST /api/clients

PUT /api/clients/{id}
DELETE /api/clients/{id}

PRODUITS

GET /api/produits
GET /api/produits/{id}
GET /api/produits/rupture
POST /api/produits
PUT /api/produits/{id}

DEVIS

GET /api/devis
GET /api/devis/{id}
POST /api/devis
PATCH /api/devis/{id}/statut/{statut}
POST /api/devis/{id}/convertir

COMMANDES

GET /api/commandes
GET /api/commandes/{id}
GET /api/commandes/site/{site}
GET /api/commandes/statut/{statut}
POST /api/commandes
PATCH /api/commandes/{id}/statut/{statut}

FACTURES

GET /api/factures
GET /api/factures/{id}
GET /api/factures/retard
POST /api/factures
POST /api/factures/paiements

STOCK

GET /api/stock/mouvements/{produitId}
GET /api/stock/rupture
POST /api/stock/mouvements

DASHBOARD

GET /api/dashboard

9. Comptes Et Donnees Demo

-
- DataInitializer cree quatre comptes si la table users est vide : admin, gerant, commercial, comptable.
 - admin@c2a.tn / admin123
 - gerant@c2a.tn / gerant123
 - commercial@c2a.tn / commercial123
 - comptable@c2a.tn / comptable123
 - Le README mentionne d autres comptes : admin@c2a.com / 123456 et vendeur@c2a.com / 123456, ce qui ne correspond pas au code actuel.

10. Risques Et Erreurs Possibles

- Critique : generation numero commande dans DevisServiceImpl utilise `String.valueOf(commandeRepository.findByNumero(prefix))`, ce qui peut produire `Optional.empty` puis casser le parsing numerique.
- Critique : `CommandeRepository.findLastNumero` utilise LIKE :prefix sans joker %, donc peut ne jamais trouver le dernier numero et provoquer des doublons.
- Critique : `DevisRepository` utilise une requete JPQL avec LIKE :prefix%, syntaxe suspecte a verifier au demarrage.
- Factures : numero genere avec `Math.random()`, risque de collision.
- Creances : la creation d une facture ne semble pas augmenter `client.soldeCreance`, mais le paiement le diminue ensuite.
- Dashboard : le frontend affiche `topClients` et `topProduits`, mais le backend ne les remplit pas.
- Stock : `AJUSTEMENT` est traite comme une sortie, pas comme une correction explicite du stock.
- Stock : `userId` est transmis par query param et non derive du JWT, donc attribution manipulable.
- Securite : `jwt.secret`, user MySQL root, password vide et configuration mail sont en clair dans `application.properties`.
- CORS : `cors.allowed-origins` est declare dans `application.properties`, mais aucun bean CORS n a ete trouve dans `SecurityConfig`.
- Frontend : toutes les pages sont visibles dans le menu pour tout utilisateur connecte ; les restrictions sont surtout backend.
- README incoherent avec le code : port 8080 vs 8081, base `c2a_ventes` vs `c2a_vente`, comptes demo differents.
- Hygiene repo : aucun `.gitignore` trouve ; `node_modules`, `dist` et `target` apparaissent comme artefacts locaux non ignores.
- Tests : aucun test backend ou frontend visible dans les sources inspectees.

11. Verification Du Plan

- Aucun changement d API.
- Aucun changement React.
- Aucun changement Spring Boot.
- Aucun changement base de donnees.
- PDF genere en francais a partir du diagnostic precedent.
- Le PDF indique explicitement qu aucune correction automatique ni modification de code n a ete lancee.

12. Conclusion

Le projet presente une architecture coherente pour un PFE full-stack, avec separation claire frontend/backend et une base metier deja avancee. Les priorites techniques a traiter avant demonstration ou livraison sont la numerotation des documents, la coherence README/configuration, la securisation des secrets, la gestion correcte des creances, le dashboard incomplet et l alignement des permissions frontend/backend.